Recursive Agent Optimization

Apurva Gandhi[1] **Satyaki Chakraborty**[2] **Xiangjun Wang**[2]
Aviral Kumar[1] **Graham Neubig**[1]
[1]Carnegie Mellon University & [2]Amazon AGI Labs
{apurvag,aviralku,gneubig}@cs.cmu.edu
{satyaki,xjai}@amazon.com

Abstract

We introduce *Recursive Agent Optimization (RAO)*, a reinforcement learning approach for training *recursive agents*: agents that can spawn and delegate sub-tasks to new instantiations of themselves recursively. Recursive agents implement an inference-time scaling algorithm that naturally allows agents to scale to longer contexts and generalize to more difficult problems via divide-and-conquer. RAO provides a method to train models to best take advantage of such recursive inference, teaching agents *when* and *how* to delegate and communicate. We find that recursive agents trained in this way enjoy better training efficiency, can scale to tasks that go beyond the model's context window, generalize to tasks much harder than the ones the agent was trained on, and can enjoy reduced wall-clock time compared to single-agent systems.[1]

1 Introduction

Large language model (LLM) agents are increasingly deployed on real-world tasks such as software engineering, research assistance, and computer use. As these systems improve, the tasks users expect them to solve are also becoming harder: they involve longer horizons, larger effective working memory, and substantial exploration and backtracking. Many such tasks are naturally amenable to divide-and-conquer. For instance, a large software change can be decomposed into code investigation and editing subproblems; a research task can be split into retrieval, synthesis, and verification stages; a long document or log corpus can be partitioned into manageable pieces and processed in parallel.

This has led to interest in *recursive* agent systems (Anthropic, 2025; OpenAI, 2026; Sun et al., 2025; Zhang et al., 2025), in which an agent can spawn sub-agents to solve subtasks with fresh contexts. Recursive execution offers several advantages. **First**, it expands effective working memory, since each sub-agent receives a fresh context window rather than inheriting the full history of its parent. **Second**, it enables divide-and-conquer by breaking a large task into smaller, more tractable subproblems. **Third**, when subproblems can be solved independently, recursion can exploit concurrency and reduce wall-clock latency for accomplishing a task. Together, these properties make recursion a compelling inference-time scaling primitive for agents.

However, most existing recursive and multi-agent systems treat recursion purely as an inference-time scaffold wrapped around a pretrained model (Prasad et al., 2024; Geng & Neubig, 2026; Wu et al., 2023). The model itself is typically not trained to decide when delegation is useful, how to formulate effective subtasks, how to communicate information across levels of the execution tree, or how to combine sub-agent outputs into a final solution. If recursive execution is going to be a core test-time primitive, then the policy should be trained to use it well. As such, prior work shows training models with inference-time scaffolds in the loop does make them more amenable to deployment with inference-time scaffolds (Wu et al., 2026; LM-Provers et al., 2026; Kimi Team et al., 2026). This raises a central question: *how should we train a model to exploit recursive inference effectively?*

[1]Webpage and code: https://apga.github.io/RAO

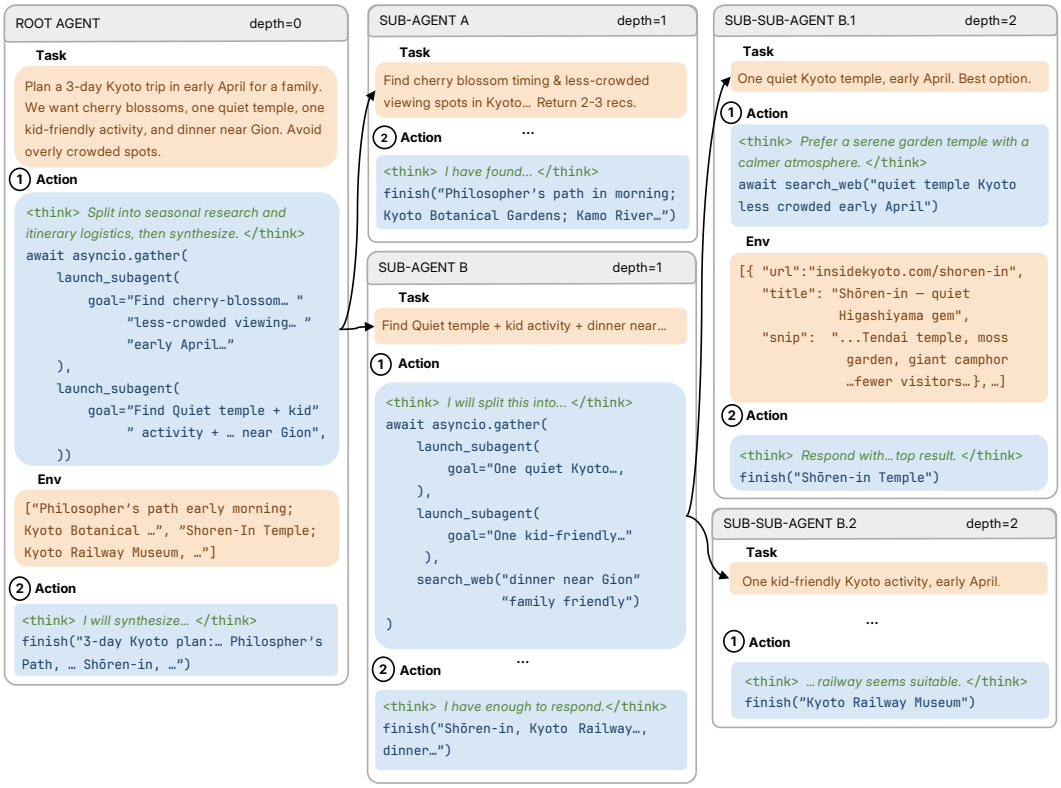


Figure 1: Example of recursive agent inference on a deep research or travel-planning task where agents can decide to spawn and delegate sub-problems to sub-agents. This produces a dynamically structured and recursive execution tree where each node corresponds to one agent instance attempting to solve an assigned task. RAO optimizes a single policy to act across all levels tree, teaching it to best take advantage of recursive inference.

In this work, we introduce *Recursive Agent Optimization (RAO)*, a reinforcement learning approach for end-to-end training of recursive agents. RAO trains a single LLM policy that is instantiated at every node of a recursively-generated execution tree. As a result, the same model must learn both how to solve the task assigned to it and how to generate useful delegated subtasks for spawned copies of itself. Importantly, RAO is not restricted to a fixed hierarchy or hand-designed orchestration scheme: it supports dynamically generated recursive execution trees and optimizes all decisions in the tree jointly. Training recursive agents may also improve learning efficiency, not just inference-time behavior. Recursive execution naturally generates related tasks at multiple levels of difficulty, yielding structured intermediate supervision and an implicit curriculum over simpler subproblems. Thus, the recursive structure used at inference time can also provide a useful training signal for generally improving model capability. This motivates a second question that we tackle: *how can we leverage the recursive structure of inference to better train agents?*

We evaluate RAO on three benchmarks, including deep research, long document processing and TEXTCRAFT-SYNTH, a controlled synthetic environment we introduce, inspired by Minecraft-style crafting tasks, in which we can vary task horizon, difficulty, and recursive structure. We find that training for recursive execution yields several benefits. Recursive agents trained with RAO solve tasks that exceed the base model's context window, generalize to substantially harder tasks than those seen during training by leveraging recursive delegation (up to 10 levels deep), and when tasks can be decomposed into parallel subproblems, can reduce wall-clock execution time relative to non-recursive baselines (up to 2.5×). In addition, RAO improves training efficiency over single-agent training by exploiting recursive decomposition during learning.

2 Recursive Agents: Inference and Training

We study *recursive agents*: agents that can delegate sub-tasks to new instances of themselves during execution. A rollout no longer resembles a single flat trajectory, but a dynamically generated tree of trajectories. While many prior works and deployed systems have explored recursive inference (Section 6), this work focuses on how to best train a model to leverage recursion: it must learn how to solve assigned tasks directly, when delegation is beneficial, how to formulate sub-tasks for child agents, and how to combine their outputs into a final solution. Unlike approaches based on fixed hierarchies or hand-specified orchestration, our setting allows the policy to generate execution trees dynamically, with arbitrary branching patterns up to a depth limit, and RAO trains all agent instances in the tree jointly.

2.1 Recursive Agent Inference

Formulation. A task X is a problem specification given to an agent. When the policy π_θ is run on X, it produces a trajectory τ_X, consisting of the sequence of actions and observations used to solve the task. During this rollout, the agent may decide to spawn zero or more child agents on delegated sub-tasks $X_1, \ldots, X_n$, where both the number and the content of these sub-tasks are chosen by the policy itself. This process induces a rooted *execution tree* $\mathcal{T}$: each node corresponds to one agent instance attempting one assigned task, the root corresponds to the original task, and the children of a node correspond to the delegated sub-tasks generated by that agent. Fig. 1 shows an example of a recursive agent trajectory on a travel-planning task. In particular, note that sub-agents can also recursively spawn their own child agents.

Implementation. We instantiate recursive agents as an extension of an agent that interleaves natural-language reasoning with code execution in a Python REPL (read-eval-print-loop) similar to Wang et al. (2024). This interface gives the model access to standard programming constructs such as variables, control flow, string manipulation, and asynchronous execution, allowing it to solve tasks by writing short programs and executing them. To support recursion, we extend the agent's action space by exposing an asynchronous function

$$\texttt{async launch_subagent(goal,...)} \rightarrow \texttt{Any}^{2},$$

which launches a new instance of the same policy on a delegated sub-task and returns the child agent's output to the parent. Because the return type is unrestricted, parents can delegate a wide range of subproblems and request outputs in whatever format is most useful for downstream computation, including structured objects rather than just strings.

The parent interacts with child agents through ordinary Python control flow. It can launch children sequentially when later sub-tasks depend on earlier results, or concurrently using standard Python libraries like `asyncio` when sub-tasks are independent. Returned objects can be stored in variables, transformed, combined, or passed into later sub-agent calls. This makes recursive decomposition highly flexible: the policy decides when to delegate, what sub-task specification to provide, what type of output to request, whether to run child agents serially or in parallel, and how to aggregate their results. To ensure bounded computation, we impose explicit limits on recursion depth and on the number of environment steps available to each agent instance. We note that our agent harness implementation resembles that of Recursive Language Models (RLMs) by Zhang et al. (2025), with the main differences being that we demonstrate positive results for recursion depths more than one, which have proven elusive in previous works (Wang, 2026), and we support concurrent sub-agent execution by implementing the delegation primitive as an asynchronous function. Furthermore, as we discuss next, the focus of this work is on training models to leverage recursion effectively at inference-time.

[2]The exact parameters and return type that the `launch_subagent` function takes can be tailored to tasks; Appendix A.3 provides examples.

2.2 Recursive Agent Optimization (RAO)

A rollout on a root task produces an execution tree whose nodes correspond to recursively instantiated copies of the same policy. RAO trains all nodes in this tree jointly. At a high level, RAO combines a local reward defined at each node with policy optimization over recursively generated execution trees.

2.2.1 Local Node Reward

A recursive agent should learn both to solve its assigned task and, when useful, to delegate productively. Ideally, we can take advantage of node-local credit assignment: a sub-agent should receive signal from whether its own assigned task and its delegated sub-tasks were solved successfully, rather than relying on the root agent's outcome-level reward. Such local signals can substantially improve credit assignment for decomposition, especially early in training when root-task success may be rare or when the initialized policy does not yet make meaningful use of delegation. At the same time, in many environments fine-grained verification is unavailable or expensive, so direct success signals for intermediate sub-agents may not always be observable.

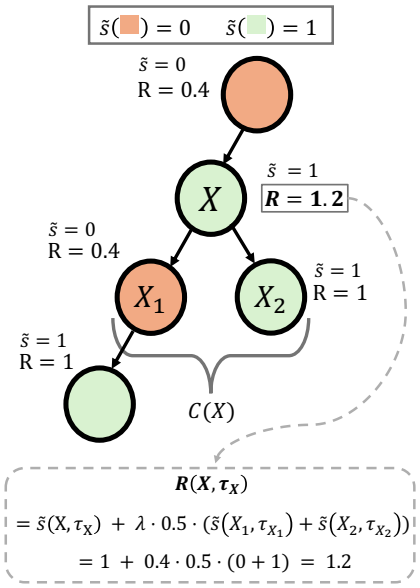


Figure 2: RAO Reward Design. Each node receives a local reward from its own success and a delegation bonus from the success rate of its children. The example uses $\lambda = 0.4$.

To accommodate both cases, we distinguish between the underlying notion of task success and the supervisory signal actually used during training each sub-agent. For a node corresponding to task X, let τ_X denote the trajectory generated while solving X, and let $C(X)$ denote the set of its immediate children. We use $\tilde{s}(X, \tau_X) \in [0, 1]$ for the success signal used in training for node X. Depending on the environment, $\tilde{s}(X, \tau_X)$ may be instantiated in different ways: it may come from exact sub-task verification, from a learned or LLM-based judge, or, when no node-local supervision is available, from a proxy such as the success of the root task. RAO's credit assignment becomes more faithful as $\tilde{s}(X, \tau_X)$ more accurately reflects the success of the corresponding node-local task X.

We define the reward for a given node as:

$$R(X, \tau_X) = \underbrace{\tilde{s}(X, \tau_X)}_{\text{success(X) / proxy}} + \underbrace{\lambda \cdot \frac{1}{|C(X)|} \sum_{c \in C(X)} \tilde{s}(c, \tau_c)}_{\text{delegation bonus}}, \tag{1}$$

with the convention that the second term is zero when $|C(X)| = 0$. Here $\lambda \geq 0$ controls the strength of the delegation bonus.

The first term rewards the agent for solving its own assigned task according to the available supervisory signal. The second term rewards it when the sub-tasks it creates are successfully completed by its children. Using the *success rate* of immediate children, rather than the raw number of successful children, avoids directly rewarding the policy for spawning more children purely to collect additional bonus. This makes the delegation bonus better aligned with the quality of delegation rather than the quantity of delegation.

The reward is local: every node, whether root, internal, or leaf, is scored using the same rule based only on its own task signal and the signals of its immediate children. Setting $\lambda = 0$ recovers a purely local-success-based reward. In practice, the delegation bonus is most useful in regimes where the initial policy under-utilizes delegation and needs additional signal to learn when and how to decompose. When the policy already delegates sufficiently

at initialization, we can simply set $\lambda = 0$ and train with the node-level success signal alone. Fig. 2 shows an example of reward computation with $\lambda = 0.4$.

2.2.2 *Policy Optimization Objective*

Recursive execution induces a family of related task distributions across depths. Let $\mathcal{D}_0$ denote the root task distribution, and let $\mathcal{D}_d(\theta)$ for $d \geq 1$ denote the distribution of depth-d sub-tasks generated by recursively applying the current policy. Training therefore optimizes a shared policy over both root tasks and policy-generated descendants:

$$J(\theta) = \sum_{d=0}^{D} \mathbb{E}_{X \sim \mathcal{D}_d(\theta)} \left[\mathbb{E}_{\tau_X \sim \pi_\theta(\cdot|X)} \left[R(X, \tau_X) \right] \right]. \tag{2}$$

This view helps explain why recursive training can be effective: the same parameters are trained across a hierarchy of related tasks, and the induced sub-tasks are often simpler or more structured than the original root task, generating a natural curriculum.

For each root task, we sample G independent recursive rollout trees $\{\mathcal{T}^{(g)}\}_{g=1}^{G}$. Let $R_{\text{root}}^{(g)}$ denote the reward of the root node in rollout g. For any trajectory τ belonging to rollout g, we define its advantage using a leave-one-out baseline over root rewards:

$$A(\tau^{(g)}) = R(\tau^{(g)}) - b_{-g}, \qquad b_{-g} = \frac{1}{G-1} \sum_{g' \neq g} R_{\text{root}}^{(g')}. \tag{3}$$

We use the same root-group baseline for all trajectories within a rollout tree, including child trajectories. Although this may not be the lowest-variance baseline, especially when each sub-task is different, it places all nodes generated under the same root task on a common reference scale and is practical by avoiding the need for a critic or constructing separate comparison groups over policy-generated sub-tasks, which generally differ across rollouts. Furthermore, a standard leave-one-out argument shows that this baseline is unbiased and hence sound; we defer the proof to Appendix A.2.

A practical issue is that recursive rollouts may contain very different numbers of trajectories at different depths. In some domains, sub-agent trajectories can greatly outnumber root trajectories, causing optimization to become dominated by deeper parts of the execution tree if all trajectories are simply pooled together. To mitigate this effect, we use *depth-level inverse-frequency weighting*, which downweights trajectories from depths that are overrepresented in the batch while preserving the overall scale of the update.

Let $\mathcal{B}_d$ denote the set of all depth-d trajectories in the batch, and let $N_d = |\mathcal{B}_d|$. We assign each trajectory at depth d a weight

$$w_d = \alpha \cdot \frac{1}{N_d}, \qquad \alpha = \frac{\sum_{d=0}^{D} N_d}{\sum_{d=0}^{D} N_d \cdot \frac{1}{N_d}}, \tag{4}$$

where α is a normalization constant chosen so that the total weight over the batch is preserved. The corresponding estimator is

$$\hat{\nabla} J(\theta) = \sum_{d=0}^{D} \sum_{\tau \in \mathcal{B}_d} w_d \, A(\tau) \, \nabla_\theta \log \pi_\theta(\tau). \tag{5}$$

Intuitively, this assigns smaller weight to trajectories from depths that appear more frequently in the batch, reducing the tendency of heavily populated levels of the tree to dominate learning, while the normalization keeps the overall update magnitude approximately unchanged.

Summary: RAO

1. Trains a **single, shared policy** to act across recursive rollouts with dynamically generated execution trees.
2. Provides **dense credit assignment** through a local reward for each node in the recursive execution tree (Eq. 1).
3. Computes advantages by comparing each node's local reward to a **leave-one-out baseline computed from root rollout rewards** (Eq. 3).
4. Optimizes a **weighted, multi-task objective** over tasks sampled from different depths of recursive rollouts, yielding a **self-induced curriculum** (Eqs. 2 and 5).

3 Experiments

We evaluate RAO on three benchmarks and give an overview below. The full action space for each of the benchmarks is provided in Appendix A.3, while additional experiment details and hyperparameters are provided in Appendix A.4.

3.1 TextCraft-Synth

To study the properties of recursive-agent training in a controlled setting, we introduce TEXTCRAFT-SYNTH, inspired by the TEXTCRAFT benchmark from Prasad et al. (2024). In TEXTCRAFT, an agent is given an initial inventory and a target item to craft using Minecraft recipes. TEXTCRAFT-SYNTH is a more challenging testbed: we expand the action space with additional APIs, such as recipe search, and replace Minecraft recipes with synthetically generated items and dependencies. This synthetic construction lets us scale the benchmark to arbitrarily deep tasks. Crafting is naturally compositional and recursive, since an item required for the target may itself need to be crafted through multiple intermediate steps. We include tasks of varying difficulty and length by controlling the depth of the underlying crafting tree: *Easy* tasks have depth 2–3, *Medium* tasks depth 4–6, and *Hard* tasks depth 7–9.

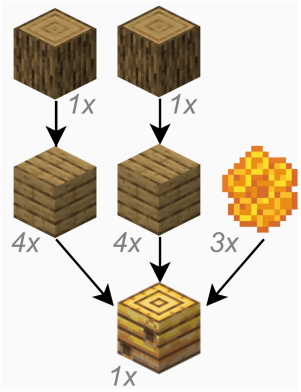


Figure 3: Figure adapted from Prasad et al. (2024) visualizing a TEXTCRAFT crafting tree. In order to craft the target beehive, the agent must first craft oak planks from oak logs.

For TEXTCRAFT-SYNTH, we study two training settings. In the *constrained-context* setting, we limit the model to an 8K context window during both training and inference to test whether RAO can exploit the effectively longer horizon enabled by recursive decomposition. In the *unconstrained* setting, we use a 40K context window during training and 256K during inference. In both settings, we train only on medium-difficulty tasks and evaluate generalization to easy and hard tasks at test time. We use Qwen-3-4B-Instruct-2507 (Bai et al., 2025) for all TEXTCRAFT-SYNTH experiments. Notably, 40K tokens is already more than sufficient for easy and medium tasks, and 256K is sufficient for all tasks; so in the unconstrained setting the single-agent baseline is not limited by context length. We do not include a delegation bonus ($\lambda = 0$) since the agent obtained at initialization from the base model is already able to solve at least a few tasks by employing recursion to some extent. We limit maximum recursion depth to 6 during training, but allow up to a depth of 12 for evaluation.

3.2 Oolong-Real

We also evaluate on OOLONG-REAL (Bertsch et al., 2025), a long-context benchmark that requires aggregating information from very long Dungeons & Dragons transcripts. We chose this setting because long-context reasoning is central to many practical agent tasks:

for example, a software agent may need to identify the cause of a failure in a very long log file, while a research agent may need to synthesize information across many documents.

For this benchmark, we train Qwen3-VL-30B-A3B-Instruct (Bai et al., 2025) with a 32K context limit. This limit is imposed by our training infrastructure: we use the Tinker service (Thinking Machines Lab, 2025), which supports at most 32K tokens for this model. Since OOLONG-REAL instances require processing documents of at least 55K tokens, a single-agent baseline cannot read the full input and must instead rely on heuristics such as regex, string matching, or selectively printing portions of the document. A recursive agent, by contrast, can process the entire input by spawning sub-agents over chunks, each with a fresh context window. This therefore provides a realistic test of whether recursive agents can overcome practical context constraints through decomposition. To speed up training, we only train on input contexts with lengths $<=$ 240K characters ($\sim$60K tokens), but evaluate on contexts up to 220K tokens. Both training and inference were constrained to a maximum recursion depth of 2 (0-indexed; 3 levels including the root agent). In contrast to the 32K training context window limit, for inference during evaluation, we use a context window of 256K tokens.

For the root-task reward, we follow the OOLONG-REAL scoring procedure, which uses partial credit for numerical answers, exact match for string answers, and overlap-based partial credit for list answers. For sub-agent success signal, unlike TEXTCRAFT-SYNTH we do not have perfect verifiers, and so instead rely on an LLM-judge using GPT-5-mini (Singh et al., 2025) (prompt provided in Appendix A.7). To help warm start training, we also use a delegation bonus with $\lambda = 0.4$. Similar to Zhang et al. (2025), we do not place the entire input directly into the agent prompt. Instead, we pre-populate the agent's Python interpreter with a `context` string that the agent can interact with programmatically. The recursive agent can then pass smaller chunks to sub-agents via `launch_subagent(goal: str, context: str)`.

3.3 DeepDive

Finally, we benchmark RAO on the deep research domain using the DEEPDIVE dataset (Lu et al., 2025). The dataset contains challenging QA pairs constructed by performing controlled walks over knowledge graphs and generating questions that require multi-hop, iterative web searches and synthesis over information scattered across the web to answer. In particular, answering DEEPDIVE questions requires breaking down the question into a chain of sequentially dependent sub-queries. A representative example is provided below:

> **Question:** A "Historic State" in Southeast Asia, known for an early dynasty, employed high administrators. One such advisor (active late 15th c.) notably forgave a royal scion who harmed his child. This scion became the state's last sovereign before its early 16th c. European conquest. This final sovereign is linked to a legend: a mystical woman on a high peak made demands of his ruling predecessor, including the sovereign's own youthful life-essence, for marriage. This legend became a film. What year was this film, detailing these conditions, released? **Answer:** 1962.

We train Qwen-3-4B-Instruct-2507 on this task with a 40K context limit while training and a 256K context window during evaluation. Both training and evaluation use a maximum recursion depth of 4. We set $\lambda = 0$, like for TEXTCRAFT-SYNTH and similar to OOLONG-REAL, we use an LLM-judge with GPT-5-mini for scoring sub-agent trajectories (prompt provided in Appendix A.8).

Beyond the delegation action, the agent has access to functions for searching the web and reading URL content[3].

4 Discussion of Experimental Results

Across all three benchmarks, recursive agents trained with RAO outperform flat single-agent baselines. On TEXTCRAFT-SYNTH, recursion improves success rates in both the

[3]We leverage Tavily APIs to implement these web related actions.

Table 1: TEXTCRAFT-SYNTH results across evaluation difficulties. Success rate (SR) is computed over the evaluation set. Steps and wall-clock time are computed over the intersection of tasks successfully solved by both methods at each difficulty.

(a) Context Window: 8K train, 8K eval

Difficulty	Method	SR	Steps	Time (s)
All	Single	0.24	16	7.1
	Recursive	**0.95**	33	9.9
Easy	Single	0.55	12	5.2
	Recursive	**1.0**	23	8.0
Medium	Single	0.17	25	11.1
	Recursive	**0.96**	52	13.6
Hard	Single	0.0	–	–
	Recursive	**0.88**	–	–

(b) Context Window: 40K train, 256K eval

Difficulty	Method	SR	Steps	Time (s)
All	Single	0.73	54	35.7
	Recursive	**0.96**	115	19.8
Easy	Single	0.97	11	6.6
	Recursive	**1.0**	21	8.8
Medium	Single	0.87	60	38.1
	Recursive	**0.98**	109	**20.9**
Hard	Single	0.20	252	180.0
	Recursive	**0.88**	694	**73.3**

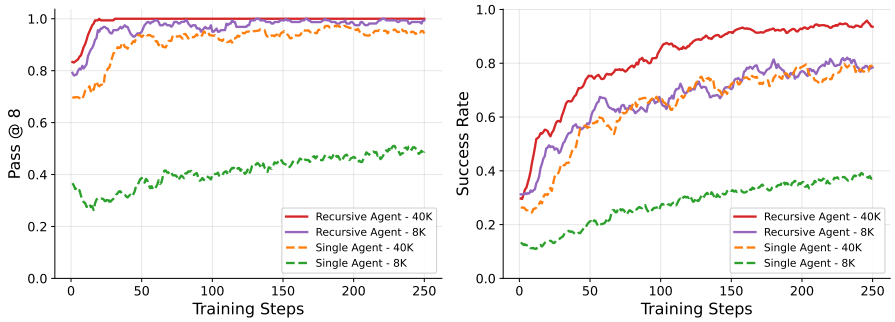


Figure 4: TEXTCRAFT-SYNTH training curvess (moving average; window size 10).

constrained-context and unconstrained-context settings, with especially large gains on hard tasks (Table 1). On OOLONG-REAL, recursive agents achieve higher average reward across all evaluated context-length buckets despite the 32K training context limit (Table 2). On DEEPDIVE, recursion improves held-out success rate from 0.24 to 0.40 (Table 3). Together, these results suggest that RAO is effective at training agents to exploit recursive execution across diverse task structures. Below, we discuss specific takeaways in detail.

Recursive agents can solve tasks that exceed the model's context window. We observe that recursive agents can solve tasks even when limiting training to an 8K context window in TEXTCRAFT-SYNTH in Fig. 4. Despite this constraint, training with RAO learns the task effectively and recovers most of the performance of models trained with 40K context. In contrast, the single-agent baseline trained with 8K context lags far behind in both success rate and pass@8. The same pattern appears in Table 1 under constrained evaluation: the single-agent baseline reaches only 24% overall success and 0% on hard tasks, whereas the recursive model with 8K context matches the unconstrained recursive model, achieving 95% overall success and 88% on hard tasks.

We observe similar evidence on OOLONG-REAL. Although these models are trained with a 32K context limit, the tasks require processing at least ∼55K tokens. Fig. 5 shows that the recursive agent leverages recursion to effectively expand its usable context and achieves a large gap over the single-agent baseline. We interestingly observe a transient failure mode around training steps 40–80, where the recursive agent briefly learns to print the entire input in the root context, exhausting its context window. It then quickly recovers by relearning the correct strategy: chunking the input and delegating smaller context chunks to sub-agents.

Recursive agents enjoy better training efficiency even when not context constrained. Even when the model is given a large enough context window to solve the task directly,

Table 2: OOLONG-REAL results across context lengths from 55K-175K. Average reward is reported on a sample of 650 bucketed into 55K-175K average input context lengths. Steps and wall-clock time are reported on the common non-zero-score intersection.

Method	Avg.	55K	118K	175K	Steps	Time (s)
Single	0.203	0.351	0.183	0.129	**7.1**	**12.6**
Recursive	**0.320**	**0.454**	**0.315**	**0.249**	61.5	175.4

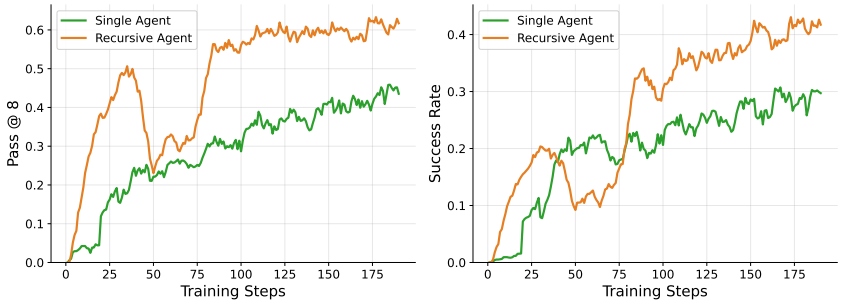


Figure 5: OOLONG-REAL training curves (moving average; window size 10).

recursive agents learn substantially faster. On TEXTCRAFT-SYNTH , Fig. 4 shows that the recursive agent trained with 40K context attains a higher pass@8 much earlier in training and maintains a strong lead in success rate over the single-agent approach. As discussed in Section 2.2, we attribute this to the structured sub-agent rewards that serve a role akin to dense process rewards and the self-generated curriculum induced by recursive execution. On DEEPDIVE (Fig. 6), we also observe that the single agent approach learns much more slowly compared to the recursive agent, ending up with a success rate gap of more than 8 percentage points after just 75 steps of training, with the trend suggesting that the gap would further increase with more training. The training performance gap also translates to the DEEPDIVE evaluation results: Table 3 shows that the recursive agent beats the single agent by 16 percentage points in success rate.

Recursive agents generalize better to harder tasks. All TEXTCRAFT-SYNTH runs are trained only on medium-difficulty problems. Table 1 shows that while both single-agent and recursive models generalize well to easy tasks, only the recursive agents perform strongly on hard tasks (88% vs. 20% success rate). We see a similar trend on OOLONG-REAL: although training uses a filtered prompt set requiring fewer than ∼60K tokens, the recursive agent maintains a substantial lead over the single agent even on inputs up to 175K tokens (Table 2).

Notably, our 30B recursive model approaches the OOLONG-REAL performance of much larger frontier models, including Claude-Sonnet-4 (0.37), o3 (0.37), and GPT-5-mini (0.35) (Bertsch et al., 2025). We hypothesize that this stronger test-time scaling comes from teaching the model a divide-and-conquer strategy that naturally transfers to harder problems.

Recursive agents can solve parallelizable tasks faster. Beyond improving capability, recursion can also reduce wall-clock time when tasks decompose into independent subtasks that can be executed concurrently. This advantage becomes more pronounced as task difficulty increases. In Table 1 for TEXTCRAFT-SYNTH, the single-agent baseline is slightly faster on easy tasks, but the recursive agent is 1.8× and 2.5× faster on medium and hard tasks, respectively, despite taking 1.8× and 2.8× more steps.

We do not observe the same advantage on OOLONG-REAL, where the single-agent baseline is 14× faster. However, this reflects a qualitative difference in strategy rather than an advantage in solving the task correctly. Because training is limited to 32K context, the single-agent baseline cannot print the full 55K+ token input in context and instead learns to rely almost entirely on fast but inaccurate programmatic heuristics such as regex and string

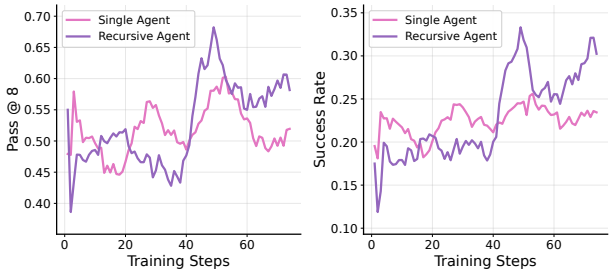


Figure 6: DEEPDIVE training curves (moving average; window size 10)

Table 3: DEEPDIVE evaluation results on 50 held-out tasks. Steps and wall-clock time are reported on tasks solved by both methods.

Method	SR	Steps	Time (s)
Single	0.24	**5.2**	**13.3**
Recursive	**0.40**	121.2	233.0

matching. The recursive agent, by contrast, learns the more reliable strategy: printing out and reading the context in chunks via sub-agents. This yields much higher accuracy, but at the cost of longer execution time.

On DEEPDIVE, while recursive agents achieve a much higher success rate, they are 18× slower on tasks solved by both methods (Table 3). This is because DEEPDIVE tasks decompose into sequentially dependent sub-tasks instead of parallelizable ones, as described in Section 3.3. In fact, while 83.9% of TEXTCRAFT-SYNTH's `launch_subagent` calls were executed concurrently, only 1.6% of DEEPDIVE delegations were concurrent.

Recursive agents adapt their delegation depth to the task. RAO also teaches agents *when* to delegate and *how much* to delegate. Fig. 7 plots the maximum depth reached by successful TEXTCRAFT-SYNTH rollouts from the recursive agent. The resulting depth distributions align closely with the task-depth patterns described in Section 3, suggesting that the agent learns difficulty-appropriate delegation behavior. Moreover, although training caps the maximum depth at 6, the agent still learns to scale to greater depths when solving hard problems.

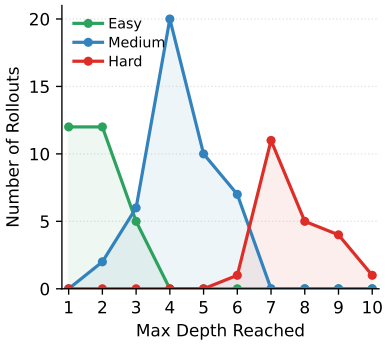


Figure 7: Maximum delegation depth on TEXTCRAFT-SYNTH.

On OOLONG-REAL, nearly all successful rollouts have maximum depth 1. This is intuitive: long-context aggregation tasks are best solved by chunking the context once and delegating each chunk to a single layer of children. On DEEPDIVE, the average, maximum depth on tasks solved by both the single agent and recursive agent is 2.9, while the depth on tasks uniquely solved by the recursive agent is 4. This suggests that the lift in success for recursive agent on DEEPDIVE stems from the fact that it allocates more test-time compute on harder tasks. Overall, RAO learns task-appropriate delegation strategies rather than applying recursion uniformly.

5 Ablation Experiments

We perform an ablation study to understand the contribution of reward design and depth-level inverse-frequency weighting in RAO. For this study, we train different variants on TEXTCRAFT-SYNTH medium problems for 100 optimization steps.

Reward Design. We compare two reward variants. In *Only Root Rewards (Sparse)*, we disable subagent success rewards and instead propagate the root-agent reward to each subagent. In *Both Root and Subagent Rewards (Dense)*, both root agents and subagents receive their own task-specific rewards.

Trajectory Weighting. We compare variants with depth-level inverse-frequency weighting enabled (*Weighted*) or disabled (*Unweighted*).

Fig. 8 shows the training curves for these experiments. Dense rewards and trajectory weighting are both important for achieving the best training efficiency and performance.

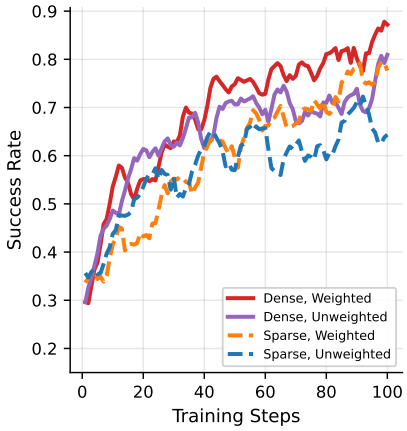


Figure 8: Ablation of RAO design choices on TEXTCRAFT-SYNTH.

6 Related Work

Recursive and multi-agent inference scaffolds. Recursive and multi-agent scaffolds are increasingly used to scale LLM agents at inference time. Deployed coding agents such as Claude Code and Codex expose subagents as a practical delegation primitive for context isolation, parallel work, and specialized tool use (Anthropic, 2025; OpenAI, 2026). Research systems such as ADaPT and THREAD support deeper recursive decomposition, allowing agents to recursively break tasks into simpler subproblems (Prasad et al., 2024; Schroeder et al., 2024). However, these systems primarily treat recursion as an inference-time scaffold around a fixed model, rather than training the model to use recursive execution effectively.

Training agents to delegate. A smaller line of recent work has begun to train models to use delegation. Our agent scaffold is closest to Recursive Language Models (RLMs), which also extend a CodeAct-style Python REPL agent (Wang et al., 2024) with a delegation function (Zhang et al., 2025; Wang et al., 2024). However, Zhang et al. (2025) primarily study depth-1 delegation as an inference-time primitive; their training experiment is small-scale, uses supervised fine-tuning, and trains only the root agent to delegate. Similarly, Kimi K2.5 trains a depth-1 agent-swarm system with but with reinforcement learning and supports parallel subagent execution; though they update only the root/orchestrator while keeping worker subagents fixed to an older checkpoint (Kimi Team et al., 2026). Context-Folding and AsyncThink train both organizers and workers with reinforcement learning, but remain depth-1 systems focused on restricted recursion patterns or narrower domains such as context management, Countdown, or mathematical reasoning (Sun et al., 2025; Chi, 2025). To our knowledge, RAO is the first to jointly study recursive agent training with all of the following: recursion beyond depth 1, end-to-end RL, asynchronous subagent execution, joint training of root agents and subagents, and evaluation on complex agentic applications.

Table 4: Comparison to recursive agent systems. Here, $D>1$ denotes experimentation with recursive execution beyond depth 1, Parallel denotes concurrent subagent execution, Joint denotes training both root agents and subagents, Real denotes evaluation on complex agentic applications.

Method	$D>1$	RL	Parallel	Joint	Real
ADaPT / THREAD	✓	✗	✗	✗	✗
RLMs	✗	✗	✗	✗	✓
Claude Code / Codex	✗	?	✓	?	✓
AsyncThink	✗	✓	✓	✓	✗
Context-Folding	✗	✓	✗	✓	✓
Kimi K2.5 (PARL)	✗	✓	✓	✗	✓
RAO	✓	✓	✓	✓	✓

Hierarchical reinforcement learning, process rewards, and automatic curricula. RAO is also connected to hierarchical reinforcement learning, which studies temporal abstraction and learned subpolicies through frameworks such as options, MAXQ, Feudal RL, and HIRO (Sutton et al., 1999; Dietterich, 2000; Dayan & Hinton, 1992; Nachum et al., 2018). Unlike classical HRL, our subtasks are expressed in natural language and generated online by the same language model policy rather than through fixed action abstractions or separate high- and low-level policies. Our use of node-local supervision also relates to process rewards and step-level verification (Lightman et al., 2023). Finally, recursively generated subtasks induce a curriculum over related task distributions, connecting RAO to automatic curriculum learning methods such as AMIGo (Campero et al., 2020). In RAO, however, the curriculum is generated cooperatively as part of solving an overall task, rather than

adversarially by a separate teacher proposing goals for a student. RAO can therefore be interpreted as combining natural-language temporal abstraction, local credit assignment over recursive trees, and shared-parameter learning across a policy-induced hierarchy of task distributions.

7 Conclusion

We introduced Recursive Agent Optimization (RAO), a reinforcement learning approach for training language-model agents to use recursive delegation as a learned inference-time primitive. Rather than wrapping a fixed model in a hand-designed recursive scaffold, RAO trains a single shared policy across all nodes of a dynamically generated execution tree. The resulting agent learns not only to solve assigned tasks, but also to decide when to delegate, how to specify useful sub-tasks, how to coordinate recursive copies of itself, and how to aggregate their outputs. Across three domains, RAO consistently unlocks capabilities that are difficult to obtain from flat single-agent execution alone, including better success rates, exploiting parallelism where possible, generalization to harder problems, and solving tasks over horizons far beyond the model's context window.

RAO also raises several immediate directions for scaling recursive agents beyond the settings studied here. With agents that can allocate computation through delegation, evaluation should move beyond final success alone and measure how efficiently agents use inference-time compute. While RAO trains a single shared model across all nodes of the recursive tree, practical systems may benefit from heterogeneous recursion, where stronger models supervise, verify, or synthesize the work of smaller specialist sub-agents, or vice-versa. Finally, because our experiments train recursion separately within each domain, an important next step is to train generalist recursive agents that transfer delegation strategies across domains. This also raises a related question about task heterogeneity within a single recursive rollout: our experiments primarily focus on settings where sub-tasks resemble smaller instances of the parent task, but many real-world tasks require delegation to qualitatively different subproblems, such as retrieval, verification, implementation, debugging, or synthesis. Understanding which environments best teach decomposition, coordination, and long-horizon planning will be crucial for building broadly capable recursive agents, and will require benchmarks that stress-test ultra-long horizons, parallelizable task structure, adaptive compute allocation, and multi-agent coordination.

More broadly, our results point to a simple principle: *inference-time scaffolds should not merely be designed around models; models should be trained to use them.* Recursion is one instance of this principle, but a particularly powerful one: it gives agents fresh context windows, divide-and-conquer structure, adaptive allocation of test-time compute, and opportunities for parallel execution. In this sense, recursion offers a path toward self-organizing agents. At the same time, as recursive methods and more general multi-agent systems continue to scale, we will need to address a practical training question: how should we design surrogate sampling procedures when full recursive or multi-agent rollouts are too costly to complete inside the RL loop? Answering this will require new training scaffolds, appropriate data mixtures, and objectives that transfer from these surrogates to challenging open-ended tasks. Designing such surrogate procedures is an important direction for future work, with the potential to improve performance while reducing latency.

Acknowledgements

We thank Amazon and Thinking Machines Lab for their very generous compute and Tinker training credits support respectively. We are also grateful to Tavily for providing us with credits used for web search APIs in our deep research experiments. Finally, we also thank many others who gave valuable feedback on earlier versions of the methods and experimental results: Pranjal Aggarwal, Zora Wang, Simran Khanuja, Amrith Setlur, Ian Wu, Daniel Fried, Andre He, Andy Liu, Atharva Naik, JY Koh, and Saujas Vaduguru at CMU, as well as Royce Cheng-Yue, Yifei Wang, Rajiv Dhawan and Sameep Tandon at Amazon. Apurva is supported by and is hugely grateful for the Amazon AI PhD Fellowship.

References

Anthropic. Subagents in the Claude Code SDK. https://code.claude.com/docs/en/agent-sdk/subagents, 2025. Accessed: 2026-05-06.

Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.

Amanda Bertsch, Adithya Pratapa, Teruko Mitamura, Graham Neubig, and Matthew R Gormley. Oolong: Evaluating long context reasoning and aggregation capabilities. *arXiv preprint arXiv:2511.02817*, 2025.

Andres Campero, Roberta Raileanu, Heinrich Küttler, Joshua B. Tenenbaum, Tim Rocktäschel, and Edward Grefenstette. Learning with AMIGo: Adversarially motivated intrinsic goals, 2020. URL https://arxiv.org/abs/2006.12122.

Aili Chen, Aonian Li, Bangwei Gong, Binyang Jiang, Bo Fei, Bo Yang, Boji Shan, Changqing Yu, Chao Wang, Cheng Zhu, et al. Minimax-m1: Scaling test-time compute efficiently with lightning attention. *arXiv preprint arXiv:2506.13585*, 2025.

Zhaoxuan Chi. The era of agentic organization: Learning to organize LLM reasoning with AsyncThink, 2025. URL https://arxiv.org/abs/2510.26658.

Kyle Corbitt. ART·E: How we built an email research agent that beats o3. OpenPipe Blog, April 2025. URL https://openpipe.ai/blog/art-e-mail-agent. Accessed: 2026-05-05.

Peter Dayan and Geoffrey E Hinton. Feudal reinforcement learning. *Advances in neural information processing systems*, 5, 1992.

T. G. Dietterich. Hierarchical reinforcement learning with the MAXQ value function decomposition. *Journal of Artificial Intelligence Research*, 13:227–303, 2000. doi: 10.1613/jair.639. URL https://doi.org/10.1613/jair.639.

Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. Areal: A large-scale asynchronous reinforcement learning system for language reasoning, 2025. URL https://arxiv.org/abs/2505.24298.

Jiayi Geng and Graham Neubig. Effective strategies for asynchronous software engineering agents. *arXiv preprint arXiv:2603.21489*, 2026.

Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, et al. Kimi k2. 5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step, 2023. URL https://arxiv.org/abs/2305.20050.

LM-Provers, Yuxiao Qu, Amrith Setlur, Jasper Dekoninck, Edward Beeching, Jia Li, Ian Wu, Lewis Tunstall, and Aviral Kumar. Qed-nano: Teaching a tiny model to prove hard theorems. https://huggingface.co/spaces/lm-provers/qed-nano-blogpost, 2026. Blog post.

Rui Lu, Zhenyu Hou, Zihan Wang, Hanchen Zhang, Xiao Liu, Yujiang Li, Shi Feng, Jie Tang, and Yuxiao Dong. Deepdive: Advancing deep search agents with knowledge graphs and multi-turn rl. *arXiv preprint arXiv:2509.10446*, 2025.

Ofir Nachum, Shixiang Gu, Honglak Lee, and Sergey Levine. Data-efficient hierarchical reinforcement learning, 2018. URL https://arxiv.org/abs/1805.08296.

OpenAI. Subagents: Use subagents and custom agents in Codex. https://developers.openai.com/codex/subagents, 2026. Accessed: 2026-05-06.

Archiki Prasad, Alexander Koller, Mareike Hartmann, Peter Clark, Ashish Sabharwal, Mohit Bansal, and Tushar Khot. ADaPT: As-needed decomposition and planning with language models. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pp. 4226–4252. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-naacl.264. URL https://aclanthology.org/2024.findings-naacl.264/.

Philip Schroeder, Nathaniel Morgan, Hongyin Luo, and James Glass. THREAD: Thinking deeper with recursive spawning, 2024. URL https://arxiv.org/abs/2405.17402.

Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*, 2025.

Weiwei Sun, Miao Lu, Zhan Ling, Kang Liu, Xuesong Yao, Yiming Yang, and Jiecao Chen. Scaling long-horizon LLM agent via context-folding, 2025. URL https://arxiv.org/abs/2510.11967.

Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999. doi: 10.1016/S0004-3702(99)00052-1. URL https://doi.org/10.1016/S0004-3702(99)00052-1.

Thinking Machines Lab. Tinker, 2025. URL https://thinkingmachines.ai/tinker/.

Daren Wang. Think, but don't overthink: Reproducing recursive language models. *arXiv preprint arXiv:2603.02615*, 2026.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents, 2024. URL https://arxiv.org/abs/2402.01030.

Ian Wu, Yuxiao Qu, Amrith Setlur, and Aviral Kumar. Reasoning cache: Continual improvement over long horizons via short-horizon rl. *arXiv preprint arXiv:2602.03773*, 2026.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen LLM applications via multi-agent conversation, 2023. URL https://arxiv.org/abs/2308.08155.

Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive language models, 2025. URL https://arxiv.org/abs/2512.24601.

A Appendix

A.1 When does RAO work best?

The main experiments in this work are performed on tasks with non-trivial difficulty and/or length (TEXTCRAFT-SYNTH, OOLONG-REAL and DEEPDIVE). On such tasks, RAO helps significantly with many advantages over single agent inference and training, as we discuss in detail in Section 4.

But what about when tasks are both easy and short-horizon? To test RAO in this setting, we train on an easier variant of deep research using the ART-E (Corbitt, 2025) dataset, where an agent needs to search over a user's emails to answer a question. In particular, unlike DEEPDIVE tasks which require numerous sequentially dependent searches to find and synthesize information scattered across the internet, ART-E tasks are much easier, requiring finding only a single relevant email in a user's inbox to answer the question.

In this setting, we find that RAO still helps the model learn more rapidly in the initial steps of training, but then later the performance equalizes between the single and recursive agents (Fig. 9). We also observe this for TEXTCRAFT-SYNTH in Table 1b where a significant gap in performance appears only for medium and higher difficulties and not on easy tasks. This is intuitive: *RAO helps the most when tasks are either difficult enough to benefit from divide-and-conquer and/or long-horizon enough to require extended context windows.*

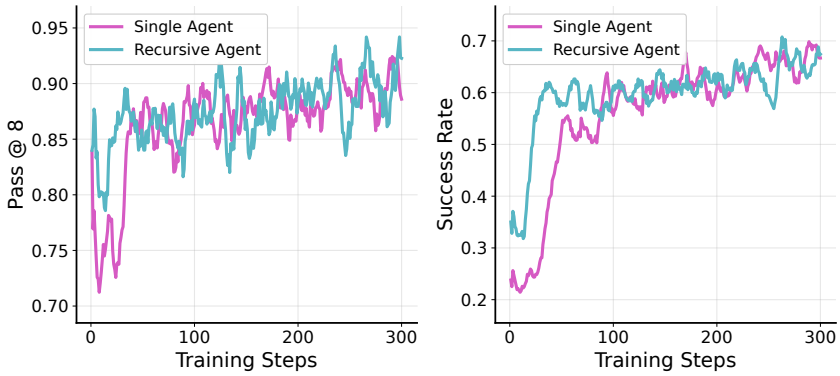


Figure 9: ART-E (Email Search) training curves for Qwen-3-14B (moving average; window size 10).

A.2 Unbiased Baseline for RAO

Lemma 1 (Unbiased leave-one-out baseline). *Let $\tau^{(g)}$ be any trajectory in rollout tree $\mathcal{T}^{(g)}$ — root or sub-agent — and let b_{-g} be the leave-one-out baseline defined in Eq. (3). Then*

$$\mathbb{E}\Big[(R(\tau^{(g)}) - b_{-g})\,\nabla_\theta \log \pi_\theta(\tau^{(g)})\Big] = \mathbb{E}\Big[R(\tau^{(g)})\,\nabla_\theta \log \pi_\theta(\tau^{(g)})\Big],$$

i.e., subtracting b_{-g} does not bias the policy gradient estimator, even when $\tau^{(g)}$ is a sub-agent trajectory.

Proof. It suffices to show that the bias term introduced by the baseline vanishes:

$$\mathbb{E}\Big[b_{-g}\,\nabla_\theta \log \pi_\theta(\tau^{(g)})\Big] = 0.$$

By construction, b_{-g} is a function of the root rewards $\{R_{\text{root}}^{(g')}\}_{g' \neq g}$ alone. Since the G rollout trees are sampled independently, b_{-g} is independent of $\tau^{(g)}$ and of the task assigned to the node that generated it, including for any sub-agent within rollout g. Therefore,

$$\mathbb{E}\Big[b_{-g}\,\nabla_\theta \log \pi_\theta(\tau^{(g)})\Big] = \mathbb{E}[b_{-g}] \cdot \mathbb{E}\Big[\nabla_\theta \log \pi_\theta(\tau^{(g)})\Big] = 0,$$

where the second factor vanishes by the score-function identity $\mathbb{E}\left[\nabla_\theta \log \pi_\theta(\tau^{(g)})\right] = 0$. □

A.3 Environment Action Spaces

For each domain, the agent has access to standard python functionality via a python REPL implemented using Jupyter's IPython kernel. On top of this, each domain has additional predefined functions available. We describe these in the tables below.

Action Type	Description
`craft(ingredients: dict, target: tuple[str, int]) -> str`	Try crafting the target given the ingredients.
`get_info(items: list) -> list[dict]`	Get information about item(s), including crafting recipes.
`finish(message: str) -> str`	Finish the rollout with a message string.
`launch_subagent(targets: dict) -> str`	Launch a subagent with some goal targets.

Table 5: TEXTCRAFT-SYNTH environment actions.

Action Type	Description
`launch_subagent(goal: str, context: str) -> Any`	Launch a subagent with a delegated goal.

Table 6: OOLONG-REAL environment actions.

Action Type	Description
`search_web(query: str, max_results: int = 5) -> dict`	Search the web using the query.
`view_webpage_content(url: str) -> str`	Get the content of a webpage.
`finish(message: str) -> str`	Finish the rollout with a message string.
`launch_subagent(goal: str) -> Any`	Launch a subagent with a delegated goal.

Table 7: DEEPDIVE environment actions. All functions are asynchronous, allowing concurrent execution.

Action Type	Description
`search_emails(query: str, max_results: int = 10) -> dict`	Search a user's inbox using the query.
`read_email(message_id: str) -> str`	Get the content of an email.
`finish(message: str) -> str`	Finish the rollout with a message string.
`launch_subagent(goal: str) -> Any`	Launch a subagent with a delegated goal.

Table 8: ART-E environment actions. All functions are asynchronous, allowing concurrent execution.

A.4 Additional Experimentation Details

TEXTCRAFT-SYNTH and DEEPDIVE experiments were performed using the AReaL training backend (Fu et al., 2025) with a learning rate of 3e-6. For the single-agent baseline, we allow 200 and 100 steps per rollout during training for the two benchmarks, respectively, and 20K steps during evaluation. For recursive agents, we allow 25 steps for both the root and sub-agents, and for DEEPDIVE we . For OOLONG-REAL experiments we use Tinker (Thinking Machines Lab, 2025) for training: LoRa with rank 32 and a learning rate of 3e-5. The single-agent baseline is allowed 50 steps per rollout, while in the recursive agent case, both root and sub-agents were allowed 15 steps. For all experiments we use a batch size of 16 tasks and a group size of 8 rollouts and use a CISPO-style objective (Chen et al., 2025). All training was done using asynchronous RL, allowing staleness of up to 3 batches. For evaluation experiments we use an 8xH200 node for inference: for TEXTCRAFT-SYNTH and DEEPDIVE, we use data parallel across all 8 GPUs with a single rollout performing inference at a time, while for OOLONG-REAL we use tensor parallel across all 8 GPUs with 5 rollouts performing inference at a time.

A.5 Token Usage Comparison

In the tables below, we compare token usage when benchmarking the recursive and single agents across the different domains. Cache read estimates are idealized, based on an assumption of perfect cache reads. On domains like OOLONG-REAL and DEEPDIVE where the single agent learns only simple and short-horizon strategies for solving the tasks, token consumption is much higher for the recursive agent which learns more effortful and long-horizon strategies that scale to harder problems. In contrast, on TEXTCRAFT-SYNTH where both the single agent and the recursive agent learn effective long-horizon strategies for solving tasks, the recursive agent actually ends up using much fewer tokens compared to the single agent (especially as we increase difficulty). This is because as task horizons increase, single agent token usage scales quadratically with the number of steps in the trajectory, since each step needs to re-encode previous history tokens (although much of this can be alleviated with KV-Caching); on the other hand, the recursive agent splits the trajectory into many shorter sub-trajectories which do not need to re-encode a long history for each new agent step performed.

Table 9: TEXTCRAFT-SYNTH token usage on the intersection of tasks solved by both single and recursive agents in the 40K/256K setting.

Diff.	Method	Input	Output	Cache-Read	Total
All	Single	1022K	6K	1021K	1029K
	Recursive	184K	12K	153K	196K
Easy	Single	24K	2K	23K	25K
	Recursive	32K	2K	26K	34K
Medium	Single	768K	8K	767K	775K
	Recursive	169K	11K	141K	181K
Hard	Single	8594K	26K	8593K	8619K
	Recursive	1153K	72K	965K	1225K

Table 10: OOLONG-REAL token usage on the intersection of tasks with non-zero score for both single and recursive agents.

Bucket	Method	Input	Output	Cache-Read	Total
All	Single	14.2K	0.9K	13.7K	15.1K
	Recursive	257.3K	67.5K	243.5K	324.8K
55K	Single	11.0K	0.7K	10.6K	11.8K
	Recursive	119.3K	27.3K	112.4K	146.6K
118K	Single	9.9K	0.9K	9.4K	10.8K
	Recursive	244.1K	64.1K	230.8K	308.2K
175K	Single	20.3K	1.0K	19.9K	21.3K
	Recursive	365.1K	98.5K	346.0K	463.6K

Table 11: DEEPDIVE token usage on the intersection evaluation tasks solved by both methods.

Method	Input	Output	Cache-Read	Total
Single	19.1K	0.8K	18.2K	19.9K
Recursive	538.4K	23.0K	499.5K	561.4K

A.6 TextCraft-Synth Prompts

TextCraft-Synth Single Agent System Prompt

You are an agent in a crafting game. Your goal is to craft items by combining ingredients. You have access to an inventory of existing ingredients, which are sufficient to craft the target items; though, you may need to craft intermediate ingredients first.

Note: If you already have one of the target items in your inventory, you should craft the requested number of the target on top of what you already have.
For example, if you already have 2 wooden_pickaxes but your goal is to craft 3, your inventory should end up with 5 wooden_pickaxes.

CRAFTING STRATEGY:
- Recipes produce fixed quantities per execution - you cannot craft arbitrary amounts
Example: If a recipe produces 2 items, you can only craft in multiples of 2 (2, 4, 6...)
- Recipe ingredients scale with the number of times you execute it Example: Recipe "2 ore $\rightarrow$ 2 items" means 2 ore for 1 execution, 4 ore for 2 executions
- Always verify what you have before claiming something is impossible
- Check your inventory and recipe information to confirm ingredient availability
- Calculate carefully: if a recipe uses 2 ingredients to make 2 items, you need exactly 2 ingredients for 2 items

You can perform actions by writing Python code blocks. You will get multiple steps to complete the task. For your current step, first briefly reason (1-3 sentences) about your strategy in`<thought> </thought>` tags, then output your code in `<code> </code>` tags.
Your code will be executed in a Jupyter notebook and the output will be shown to you.

TextCraft-Synth Recursive Agent System Prompt

You are an agent in a crafting game.
Your goal is to craft items by combining ingredients.
You have access to an inventory of existing ingredients, which are sufficient to craft the target items; though, you may need to craft intermediate ingredients first.

Note: If you already have one of the target items in your inventory, you should craft the requested number of the target on top of what you already have.
For example, if you already have 2 wooden_pickaxes but your goal is to craft 3, your inventory should end up with 5 wooden_pickaxes.

CRAFTING STRATEGY:
- Recipes produce fixed quantities per execution - you cannot craft arbitrary amounts
Example: If a recipe produces 2 items, you can only craft in multiples of 2 (2, 4, 6...)
- Recipe ingredients scale with the number of times you execute it Example: Recipe "2 ore $\rightarrow$ 2 items" means 2 ore for 1 execution, 4 ore for 2 executions
- Always verify what you have before claiming something is impossible
- Check your inventory and recipe information to confirm ingredient availability
- Calculate carefully: if a recipe uses 2 ingredients to make 2 items, you need

exactly 2 ingredients for 2 items

DELEGATION STRATEGY:
- It is highly recommended to delegate crafting of intermediate ingredients
- Break complex tasks into INDEPENDENT subtasks that can be solved separately
- For tasks that are sufficiently complex, it is recommended to recursively delegate; i.e., subagents can further delegate to other subagents.
- Delegate one group of related items at a time, not everything at once
- Use crafting depth from get_info() to estimate budget requirements:
* crafting_depth indicates complexity (0=base item, 1=direct craft, 2+=needs intermediates)
* Budget heuristic: depth × 8-10 steps (depth=4 needs 32-40 steps, depth=8 needs 64-80 steps)
* Always check crafting_depth before delegating to avoid under-budgeting - Items can be delegated in parallel if they don't depend on each other
- Reserve budget for yourself to do final assembly after subtasks complete
- Delegated tasks share your inventory - results are immediately available
- IMPORTANT: launch_subagent is an async function, you MUST use await:
* CORRECT: await launch_subagent("item": 1, 20)
* CORRECT: await asyncio.gather(launch_subagent(...), launch_subagent(...))
* WRONG: launch_subagent("item": 1, 20) – missing await, will error

You can perform actions by writing Python code blocks. You will get multiple steps to complete the task. For your current step, first briefly reason (1-3 sentences) about your strategy in `<thought> </thought>` tags, then output your code in `<code> </code>` tags.
Your code will be executed in a Jupyter notebook and the output will be shown to you.

TextCraft-Synth User Prompt (Same for both)

Craft the following items: `<count>x <item>[, <count>x <item>, ...]`
Example:
Craft the following items: `1x m5_i3, 2x c3_i2`

TextCraft-Synth Single Agent Action Space

```
1. def craft(ingredients: dict, target: tuple[str, int]) -> str
   Craft items using ingredients from your inventory.
   - ingredients: Dict of item_name: count to consume
   - target: (item_name, total_count) where total_count must be
       divisible by recipe result_count
   - Example: craft({"m0_i1": 2, "m1_i1": 1}, ("m2_i2", 2))

2. def get_info(items: list) -> list[dict]
   Get recipe information for items.
   - Returns: List with {"item": str, "can_craft": bool,
       "is_base": bool, "in_inventory": int, "crafting_depth": int,
       "recipes": [...]}
   - crafting_depth indicates complexity: 0=base item,
       1=direct craft, 2+=needs intermediate steps
   - Each recipe shows {"ingredients": {...}, "result_count": int}
   - Example: get_info(["m2_i2", "raw_m0"])
```

```
3. def view_inventory() -> dict
   View your current inventory.
   - Returns: Dict of {item_name: count}
   - Example: inv = view_inventory()

4. def finish(message: str) -> str
   Complete the task.
   - Example: finish("Successfully crafted all required items")
```

TextCraft-Synth Recursive Agent Action Space

```
1. def craft(ingredients: dict, target: tuple[str, int]) -> str
   Craft items using ingredients from your inventory.
   - ingredients: Dict of {item_name: count} to consume
   - target: (item_name, total_count) where total_count must be
       divisible by recipe result_count
   - Example: craft({"m0_i1": 2, "m1_i1": 1}, ("m2_i2", 2))

2. def get_info(items: list) -> list[dict]
   Get recipe information for items.
   - Returns: List with {"item": str, "can_craft": bool, "is_base": bool,
       "in_inventory": int, "crafting_depth": int, "recipes": [...]}
   - crafting_depth indicates complexity: 0=base item, 1=direct craft,
       2+=needs intermediate steps
   - Each recipe shows {"ingredients": {...}, "result_count": int}
   - Example: get_info(["m2_i2", "raw_m0"])

3. def view_inventory() -> dict
   View your current inventory.
   - Returns: Dict of {item_name: count}
   - Example: inv = view_inventory()

4. def finish(message: str) -> str
   Complete the task.
   - Example: finish("Successfully crafted all required items")

5. async def launch_subagent(targets: dict, num_steps: int,
    context: str = "") -> str
   Launch a subagent to craft specific targets (shares your inventory).
   - targets: Dict of {item_name: count} to craft
   - num_steps: Budget for subagent
     * Use crafting_depth from get_info() to estimate: depth × 8-10 steps
     * Example: depth=4 needs ~32-40 steps, depth=8 needs ~64-80 steps
   - context: Optional context string for the subagent
   - Sequential: await launch_subagent({"m0_i2": 4}, 20)
   - Parallel: results = await asyncio.gather(
       launch_subagent({"m0_i2": 4}, 20),
       launch_subagent({"c1_i2": 3}, 15)
     )
   - Returns: Subagent's finish message (or list if using gather)

Note: asyncio is already imported.
Use await asyncio.gather(...) to run subtasks in parallel or
await launch_subagent() for a single subtask. Do not forget
```

```
to await the results.
```

A.7 Oolong-Real Prompts

Oolong-Real Single Agent System Prompt

You are tasked with answering a query that requires analyzing and aggregating information from a large context.

You have access to a REPL environment with the following pre-loaded variable:
- ‘context‘ (str): The full text context to analyze (may be very large)

<TIPS>
CONTEXT ANALYSIS:
- If the length on the context is very large (¿32K characters), first examine/peek into the structure of the context (what format is the data in?)
- The context may be structured in a way that the task can be solved via programmatic parsing or matching. **Only take this approach if you are 100- For the majority of cases, you should print out the context with ‘print(context)‘ to observe and read it manually and answer the question by reading the context.
ANSWER SUBMISSION:
- You can submit your answer using the ‘finish‘ function in the format requested in the user provided goal.
</TIPS>

You can perform actions by writing Python code blocks. You will get multiple steps to complete the task. For your current step, first briefly reason (1-3 sentences) about your strategy in <thought> </thought> tags, then output your code in <python> </python> tags.

Your code will be executed in a Jupyter notebook and the output will be shown to you.

Oolong-Real Recursive Agent System Prompt

You are tasked with answering a query that requires analyzing and aggregating information from a large context.
You have access to a REPL environment with the following pre-loaded variable:
- ‘context‘ (str): The full text context to analyze (may be very large)

<TIPS>
CONTEXT ANALYSIS:
- First check if the length on the context is very large (>32K characters) using ‘len(context)‘.
- For very large contexts (i.e., >32K characters), work with chunks rather than the entire context at once.
- Use subagents to process chunks and then aggregate the results to produce a final answer. Try not to split the context into too many chunks (32K characters per chunk is a good rule of thumb)
- If the context <= 32K characters, prefer to process your context by printing out and reading it rather than using programmatic heuristics.
- **IMPORTANT: DO NOT USE regex, string matching, etc. types of programmatic

heuristics. It is important to read the context with 'print(context)' to be accurate in your answer.**
SUBAGENT DELEGATION:
-**IMPORTANT: Do not use subagents if the context you need to process is <= 32K characters. Just print out the context to observe it directly and answer the question by reading the context.**
- You have the ability to spawn subagents (other instantiations of yourself), by providing them with their own 'context'/chunk to process and a goal/instruction for what result it should return.
- You can use 'asyncio.gather' to process multiple chunks simultaneously.
- Be specific about the format and type in which you expect subagents to return their results.
- Do not provide the context/chunk as part of the goal. Instead, pass it explicitly as the 'context' argument to the 'launch_subagent' function.
ANSWER SUBMISSION: - You can submit your answer using the 'finish' function in the format requested in the user provided goal.
</TIPS>

You can perform printing out or peaking into the context or launching sub-agents using Python code blocks. You will get multiple steps to complete the task. For your current step, first briefly reason (1-3 sentences) about your recursive strategy in <thought> </thought> tags, then output your code in <python> </python> tags.

Your code will be executed in a Jupyter notebook and the output will be shown to you. The python code block should be formatted as follows: <python>code block</python> without any other tags.

Do not output anything else except for <thought>...</thought>
<python>...</python>

Oolong-Real User Prompt (same for both)

<question from the dataset>
Notes:
- The task/user prompt is exactly the dataset question.
- The long document to analyze is provided separately as the pre-loaded REPL variable 'context'.

Oolong-Real Single Agent Action Space

```
Available Actions (python functions):

Pre-loaded variable:
- context (str): The full text context to analyze

1. def finish(message: str) -> str
   Complete the task with your answer.
```

Oolong-Real Recursive Agent Action Space

```
Available Actions (python functions):

Pre-loaded variable:
- context (str): The full text context to analyze

1. async def launch_subagent(goal: str, context: str) -> Any
   Launch a subagent to process a chunk of the context.
   Returns the result of the subagent's execution with return type
   specified in the goal
   (if specified, else str).
   - goal: The goal/instruction for the subagent. Tell the subagent what
    information you want and specify the exact format and type in which
    you expect the subagent to return its result.
   - context: The chunk of the context to process.

   Example:
   result = await launch_subagent(
      goal="Find the last two spells mentioned in the context by printing
       out the context and manually reading it. Return the spells as a
       stringified list of spells that I can parse with json.loads()",
       context=context_chunk
   )
   print(f"Subagent result: {result}")

  Note: `asyncio` is already imported. Use `await asyncio.gather(...)` to
  run subtasks in parallel or `await launch_subagent()` for a single subtask.
   Do not forget to await the function call.

2. def finish(result: Any) -> Any
   Complete the task with your result.
```

Oolong-Real LLM Judge Prompt for sub-agent rewards

We need to judge the performance of an agent on a task.
The task relates to aggregating some information from a potentially very large context.
1. Read the context carefully and see if the agent's answer is accurate.
2. Do not mark the agent as successful unless it prints out the context and reads it manually or alternatively uses subagents to answer the question.
For example, if the agent uses regex or string matching/contains logic to answer the question, this is a heuristic that may not be reliable in general and thus should not be marked as successful.

Using subagents is okay since subagents can view the context on behalf of the agent.
Please provide a reason and success flag (boolean value) in the following format:

```
{
  "reason": "Brief reasoning for success flag here.",
  "success": "True" or "False",
}
```

A.8 DeepDive Prompts

DeepDive Single Agent System Prompt

You are a deep research agent solving a factual question by searching the web.

You have access to Python plus web-search tools. Use them deliberately:
- Start broad, then refine based on what you learn.
- Cross-check key claims across multiple sources when possible.
- Use `await view_webpage_content(url)` when snippets are insufficient or you need detailed evidence.
- Avoid dumping huge webpage bodies into the notebook unless necessary.
- Keep intermediate notes concise and use Python to organize findings.

ANSWER SUBMISSION:
- When you are confident, call `finish(...)`.
- The final answer should directly answer the question and stay concise unless the task explicitly asks for more detail.

OTHER TIPS:
- **All functions except for finish are async functions. YOU MUST AWAIT THE RESULTS OF THESE FUNCTIONS**

You can perform actions by writing Python code blocks. You will get multiple steps to complete the task. For your current step, first briefly reason (1-3 sentences) in `<thought> </thought>` tags, then output code in `<python> </python>` tags. Your code will be executed in a Jupyter notebook and the output will be shown to you.

DeepDive Recursive Agent System Prompt

You are a deep research agent solving a factual question by searching the web.

You have access to Python plus web-search tools, and you can delegate subproblems to subagents.

RESEARCH STRATEGY:
- Break the question into a small number of meaningful subquestions.
- Search broadly first, then narrow onto the most promising sources.
- Cross-check important claims across multiple sources when possible.
- Use `await view_webpage_content(url)` when search snippets are not enough.
- Use Python to store notes, compare evidence, and synthesize findings.

DELEGATION STRATEGY:
- You have the ability to spawn subagents and delegate subtasks to them. Make effective use of subagents to solve the task!
- Use `await launch_subagent(goal)` for coherent subproblems such as source discovery, fact verification, or answering one component of a multi-hop question.
- Tell subagents exactly what to return, including format when useful.
- Subagents can run in parallel with `await asyncio.gather(...)`.
- Subagents can themselves delegate recursively.

ANSWER SUBMISSION:
- When you are confident, call `finish(...)`.
- The final answer should directly answer the question and stay concise unless the

task explicitly asks for more detail.

OTHER TIPS:
- All functions except for finish are async functions. YOU MUST AWAIT THE RESULTS OF THESE FUNCTIONS

You can perform actions by writing Python code blocks. You will get multiple steps to complete the task. For your current step, first briefly reason (1-3 sentences) about your research or delegation strategy in <thought> </thought> tags, then output code in <python> </python> tags.
Your code will be executed in a Jupyter notebook and the output will be shown to you.

DeepDive User Prompt (same for both)

<question from the dataset>

Notes:
- The task/user prompt is exactly the dataset question.
- The ground-truth answer is used only for evaluation and is not shown to the agent.

DeepDive Single Agent Action Space

```
Available Actions (python functions):

1. async def search_web(query: str, max_results: int = 5) -> dict
   Search the web for information related to the query.
   - query: The query to search for.
   - max_results: Optional maximum number of results to return.
     Must be between 1 and 20. Defaults to 5.

   Returns a dictionary containing:
   {
       "query": str,
       "follow_up_questions": list[str],
       "answer": str,
       "images": list[str],
       "results": list[dict],
       "response_time": float,
       "request_id": str,
   }

   Each result contains:
   {
       "url": str,
       "title": str,
       "content": str,
       "score": float,
       "raw_content": str | None,
   }

2. async def view_webpage_content(url: str) -> str
```

```
   View the content of a webpage.
   - url: The URL of the webpage to view.

  Returns a string containing the webpage content. This may be very long,
  so it is often useful to inspect its size before printing the full text.

3. def finish(message: str) -> str
   Complete the task with your answer.
   This is synchronous and should not be awaited.
```

DeepDive Recursive Agent Action Space

```
Available Actions (python functions):

1. async def launch_subagent(goal: str) -> Any
   Launch a subagent to solve a subtask.
   - goal: The instruction for the subagent. This can be a simple or
     compound task. Subagents can recursively delegate tasks to other
     subagents.
   - Specify the format and type of answer you expect from the subagent.

   Example:
   ps5_price_range = launch_subagent(
       "Find the price range of a PS5 across sony, bestbuy, amazon and
      gamestop. Return the answer as a string of the form '$$$ - $$$$'."
   )
   switch2_price_range = launch_subagent(
       "Find the price range of a Switch 2 across nintendo, bestbuy,
       amazon and gamestop. Return the answer as a string of the form
       '$$$ - $$$$'."
   )
   results = await asyncio.gather(
       ps5_price_range,
       switch2_price_range,
       return_exceptions=True,
   )

  Note: asyncio is already imported. Use await asyncio.gather(...) to run
   subagents in parallel or await launch_subagent(goal) for a single
   subagent. Do not forget to await the results.

2. async def search_web(query: str, max_results: int = 5) -> dict
   Search the web for information related to the query.
   - query: The query to search for.
   - max_results: Optional maximum number of results to return.
     Must be between 1 and 20. Defaults to 5.

3. async def view_webpage_content(url: str) -> str
   View the content of a webpage.
  Returns a string containing the webpage content. This may be very long.

4. def finish(message: str) -> str
   Complete the task with your answer.
   This is synchronous and should not be awaited.
```

DeepDive LLM Judge Prompt (Root Task)

We need to judge the performance of a deep research agent on a task. The task requires searching the web for information across various sources and synthesizing information together to answer a question.
The agent may use subagents to solve parts of the task. Do not penalize the model for relying on subagents, unless the subtasks delegated to the subagents are not meaningful or useful for the task.
You will be given the ground truth answer to the task and the agent's answer to the task.
When comparing the agent's answer to the ground truth answer, it is acceptable to have minor formatting differences as long as the core information is equivalent.
Please provide a reason and success flag (boolean value) in the following format:

```json
{
    "reason": "Brief reasoning for success flag here.",
    "success": true or false
}
```

DeepDive LLM Judge Prompt (Sub-Task)

We need to judge the performance of a deep research agent on a sub-task. The sub-task is part of a larger task that requires searching the web for information across various sources and synthesizing information together to answer a question.
The agent may use subagents to solve parts of the sub-task. Do not penalize the model for relying on subagents when the delegated subtasks are meaningful and useful.
You should mark the sub-task as successful if the agent's final answer is correct and, if the agent delegates, its delegation strategy is useful and efficient.
Reward delegation only when it is genuinely useful: the subtasks should be concrete, non-overlapping, and should help the agent search, verify, or synthesize information more effectively.
Mark the sub-task as unsuccessful for degenerate delegation behavior even if the final answer is correct. Degenerate behavior includes repeatedly forwarding nearly the same or whole goal to subagents without meaningful decomposition, search, or reading work, or wasteful failed launches caused by trying to delegate past the depth limit.
It is acceptable for the agent to do all the work itself, part of the work itself, or to use subagents heavily if those subagents do distinct useful work.
Do not give credit just because the wording of child tasks changes slightly from the agent's own goal; judge whether the decomposition is actually useful and efficient.
Please provide a reason and success flag (boolean value) in the following format:

```json
{
    "reason": "Brief reasoning for success flag here.",
    "success": true or false
}
```